□本介绍 - Doom - 灵雀云

Menu

本页概览 ›

□本介绍

Doom □基于 rspress↗ 开发的、适用于 Alauda 内部使用的文档开发工具，它通过实现丰富的内置插件方便文档用户开箱即用。

目录

基础能力

基于 Markdown 及其扩展 MDX

开始尝试

□础能力

- 自动生成可配置权重（顺序）的左侧侧边栏
- 静态文档全文搜索
- 多语言支持

□于 **Markdown** 及其扩展 **MDX**

MDX □一种功能强大的内容开发方式。你不仅仅可以像往常一样编写 Markdown 文件，而且可以在 Markdown 的内容中使用 React 组件：

// docs/index.mdx
import { CustomComponent } from './custom';
Hello World
<CustomComponent />

□□□□□□□□□□ MDX□ □□↗□

□□□□

□□□□□□□□□□□□ Doom □□

□□

□□

□□□□

□□□□□

□□

□□

□□ `doom` □□□□

□□□□

□□□□

API □□□□

□□□□□□□□

□□□□□□

□□□□□□

□□□□□

□□□□□□□□

□□□□□□□□□

□□□□□□□□

`sites.yaml` □□

□□□□

□□□□□□□□□

□□□□□□

□□□□□□

Algolia □□□□

Sitemap □□

□□

□□“□□□□□□□”□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

□□□□

□□□

□□

□□

Markdown

Callouts

Mermaid

□□□□

MDX

□□ MDX □□□□□□□□□□□□□□□□□

rspress □□

doom □□

□□□□□□□

□□□

□□□□□□□□□□□□□□□

`i18n.json`

`.ts/.tsx`

`.mdx`

API □□

□□ API

CRD

□□□□

□□ openapi □□

□□□□□□

`props`

□□

□□□□

□□□□□□□

□□

□□□□□□□□□□□□□□□□□□□□□ ACP □□

□□□□□□

□□□□□□

□□□□□□□□

□□□□□□□□□□

开始 - Doom - 灵雀云

Menu

本页概览 >

开始

目录

- 创建项目
- 命令行工具
  - 启动开发服务
  - 生产环境构建
  - 本地预览
  - 使用脚手架模板
  - 翻译文档
  - 导出 PDF
  - 文档检查

创建项目

首先，你可以通过以下命令创建一个新目录：

```
mkdir my-docs && cd my-docs
```

执行 `npm init -y` 来初始化一个项目。你可以使用 npm、yarn 或 pnpm 安装 doom:

npm yarn pnpm bun

```
npm install -D @alauda/doom typescript
```

然后通过如下命令创建文件：

□始 - Doom - 灵雀云

```bash
# □建 docs 目录，默认支持中英文双语
mkdir docs/en && echo '# Hello World' > docs/en/index.md
mkdir docs/zh && echo '# 你好世界' > docs/zh/index.md
```

在 `package.json` 中加上如下的脚本：

```json
{
  "scripts": {
    "dev": "doom dev",
    "build": "doom build",
    "new": "doom new",
    "serve": "doom serve",
    "translate": "doom translate",
    "export": "doom export"
  }
}
```

然后初始化一个配置文件 `doom.config.yml`：

```yaml
title: My Docs
```

同时新建 `tsconfig.json`，内容如下：

□始 - Doom - 灵雀云

```json
{
  "compilerOptions": {
    "jsx": "react-jsx",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "resolveJsonModule": true,
    "skipLibCheck": true,
    "strict": true,
    "target": "ESNext",
  },
  "mdx": {
    "checkMdx": true,
  },
}
```

最后□建 `global.d.ts` 文件，内容如下：

```ts
/// <reference types="@alauda/doom/runtime" />
```

这样你便可以在 `.mdx` 文件中类型安全地使用 doom 提供的全局组件了。

命令行工具

```
doom -h

# output
Usage: doom [options] [command]

Doctor Doom making docs.

Options:
  -V, --version                    output the version number
  -c, --config <config>            Specify the path to the config file
  -v <version>                     Specify the version of the documentation, c
  -b, --base <base>                Override the base of the documentation
  -p, --prefix <prefix>            Specify the prefix of the documentation bas
  -f, --force [boolean]            Force to
                                   1. fetch latest reference remotes or scaffo
                                   2. translate ignore hash equality check and
  -i, --ignore [boolean]           Ignore internal routes (default: false)
  -d, --download [boolean]         Display download pdf link on nav bar (defau
  -e, --export [boolean]           Run or build in exporting PDF mode, `apis/*
  -I, --include <language...>      Include **only** the specific language(s),
  -E, --exclude <language...>      Include all languages except the specific l
  -o, --out-dir <path>             Override the `outDir` defined in the config
  -r, --redirect <enum>            Whether to redirect to the locale closest t
  -R, --edit-repo [boolean|url]    Whether to enable or override the `editRepo
  -a, --algolia [boolean|alauda]   Whether to enable or use the alauda (docs.a
  -S, --site-url                   Whether to enable the siteUrl for sitemap g
  -n, --no-open                    Do not open the browser after starting the
  -h, --help                       display help for command

Commands:
  dev [options] [root]             Start the development server
  build [root]                     Build the documentation
  preview|serve [options] [root]   Preview the built documentation
  new [template]                   Generate scaffolding from templates
  translate [options] [root]       Translate the documentation
  export [options] [root]          Export the documentation as PDF, `apis/**`
  lint [root]                      Lint the documentation
  help [command]                   display help for command
```

□□□□□□□□□□

□始 - Doom - 灵雀云

启动□发服务

执行 `yarn dev` 启动开发服务，浏览器会自动打开文档首页

```
doom dev -h

# output
Usage: doom dev [options] [root]

Start the development server

Arguments:
  root                      Root directory of the documentation

Options:
  -H, --host [host]         Dev server host name
  -P, --port [port]         Dev server port number
  -l, --lazy [boolean]      Whether to enable `lazyCompilation` which could i
  --no-lazy                 Do not enable `lazyCompilation`
  -h, --help                display help for command
```

生产环境构建

执行 `yarn build` 构建生产环境代码，构建完成后会在 `dist` 目录生成静态文件

本地预览

执行 `yarn serve` 预览构建后的静态文件，注意如果使用了 `-b`，`-p` 参数构建，预览时也需要使用 `-b`，`-p` 参数

使用脚手架模板

执行 `yarn new` 使用脚手架模板生成项目、模块或文档

翻译文档

```
doom translate -h

# output
Usage: doom translate [options] [root]

Translate the documentation

Arguments:
  root                          Root directory of the documentation

Options:
  -s, --source <language>   Document source language, one of en, zh, ru (defau
  -t, --target <language>   Document target language, one of en, zh, ru (defau
  -g, --glob <path...>      Glob patterns of source dirs/files to translate
  -C, --copy [boolean]      Wether to copy relative assets to the target direc
  -h, --help                display help for command
```

- `-g, --glob` □□□□□□□□□□□□□□□□□□□□□□□□ `glob` □□□□□□□□□□□□□□□□□ □□□□□□□□□□□□□□□□

  1. `yarn translate -g abc xyz` □□□ `<root>/<source>/abc`, `<root>/<source>/xyz` □□□□□□□□□□□ `<root>/<target>/abc`, `<root>/<target>/xyz` □□□
  2. `yarn translate -g '*'` □□□ `<root>/<source>` □□□□□□□□

- `-C, --copy` □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ `false` □□□□ □□□□□□□□□□□□□□□□□□□

  - □□□□□□

    1. `/<source>/abc.jpg` □□□□□□ `<root>/public/<source>/abc.jpg` □ `<root>/public/<target>/abc.jpg`□□□□□□□□□□□□□ `/<target>/abc.jpg`
    2. `<root>/<source>/abc.mdx` □□□□ `./assets/xyz.jpg` □□□□□□ `<root>/<source>/assets/xyz.jpg` □ `<root>/<target>/assets/xyz.jpg` □□□□ □□□□□□□
    3. `<root>/<source>/abc.mdx` □□□□ `./assets/<source>/xyz.jpg` □□□□□□ `<root>/<source>/assets/<source>/xyz.jpg` □ `<root>/<target>/assets/<target>/xyz.jpg` □□□□□□□□□□□□□ `./assets/<target>/xyz.jpg`

- □□□□□□□□□□

  1. `/<source>/abc.jpg` □□□□□ `<root>/public/<target>/abc.jpg` □□□□□□□□□ □□□□□□□□□ `/<target>/abc.jpg` □□□□□□□□□□□□□□□□□
  2. `<root>/<source>/abc.mdx` □□□□ `./assets/<source>/xyz.jpg` □□□□□□ `<root>/<target>/assets/<target>/xyz.jpg` □□□□□□□□□□□□□□□□□□ `./assets/<target>/xyz.jpg` □□□□□□□□ `../<source>/assets/<target>/xyz.jpg`

WARNING

□□□□□□□□□ `-g '*'` □□□□□□□□□□□□ `source` □ `target` □□□□□□□□□ `internalRoutes` □□□□□□□□ `target` □□□□□□□□□

TIP

□□□□□□□□□□□ `AZURE_OPENAI_API_KEY` □□□□□□□□□□□□□ Leader □□

□□□□□□□□□□□□□□□□□□

```yaml
i18n:
  title:
    en: DevOps Connectors
  additionalPrompts: '□□□□ Connectors □□□□□□□□□□'
  disableAutoTranslation: false
title: DevOps □□□
```

□□□□□□□□□□□□

□□ PDF

WARNING

□□□□□□□□□□□□□ `yarn build` □□□□

```
doom export -h

# output
Usage: doom export [options] [root]

Export the documentation as PDF, `apis/**` and `*/apis/**` routes will be ign

Arguments:
  root                Root directory of the documentation

Options:
  -H, --host [host]  Serve host name
  -P, --port [port]  Serve port number (default: "4173")
  -h, --help         display help for command
```

□□ `yarn export` □□□□□ PDF □□□□□□□□□□□ `-b`, `-p` □□□□□□□□□□□□□□ `-b`, `-p` □□

□□□□□□ `playwright` ↗□□□□□□□ `build-harbor.alauda.cn/frontend/playwright-runner:doom` □□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□□□□□□□

.env.yarn

```
PLAYWRIGHT_DOWNLOAD_HOST="https://cdn.npmmirror.com/binaries/playwright"
```

□□□□□□□□□□□ pdf □□□□ `doom` □□□□□□□□□□□□ pdf □□□□□□□□□□□□□□□□

□□□□

开始 - Doom - 灵雀云

```
doom lint -h

# output
Usage: doom lint [options] [root]

Lint the documentation

Arguments:
  root                    Root directory of the documentation

Options:
  -g, --glob <path...>  Glob patterns of source dirs/files to lint (default:
  --no-cspell           Disable cspell linting
  -h, --help            display help for command
```

`doom lint` 基于 `ESLint` ↗ 和 `cspell` ↗，如果希望在编辑器中拥有更好的体验，可以安装相应的插件 ESLint ↗ / CSpell ↗，然后创建相应的配置文件：

- `eslint.config.mjs`:

  ```
  import doom from '@alauda/doom/eslint'

  export default doom(new URL('docs', import.meta.url))
  ```

- `cspell.config.mjs`:

  ```
  export { default } from '@alauda/doom/cspell'
  ```

同时我们约定当前工作目录（`CWD`）下 `.cspell` 文件夹用于存放 CSpell 的字典文件，例如你可以创建 `.cspell/k8s.txt`:

```
k8s
kubernetes
```

更多配置请参考文档检查配置

□用 - Doom - 灵雀云

Menu

使用

□置

- □置 `doom` 文档工具
- □置文件
- 基础□置
- API 文档□置
- 权限说明文档□置
- 引用文档□置
- 发行说明□置
- 左导航□置
- 内部文档路由□置
- 仅包含文档路由□置
- 语言高亮插件□置
- `sites.yaml` □置
- 翻译□置
- 在代码仓库编辑文档
- 文档导出□置
- 文档检查□置
- Algolia 搜索□置
- Sitemap □置

□用 - Doom - 灵雀云

约定

基于“约定大于配置”的理念，我们约定文档的组织方式以自动生成左侧边栏等相关内容

目录结构

元数据

排序

预览

Markdown

Callouts

Mermaid

属性扩展

MDX

使用 MDX 可以实现动态内容的展示和内容复用

rspress 组件

doom 组件

自定义组件复用

国际化

在可复用组件中使用国际化文本

`i18n.json`

`.ts/.tsx`

`.mdx`

□用 - Doom - 灵雀云

API 文档

高级 API

CRD

公共引用

指定 openapi 路径

权限说明文档

`props`

示例

引用文档

引用文档□置

部署

文档项目开发完成后我们可以将项目部署到 ACP 平台

构建与预览

多版本构建

合并目录结构

动态挂载□置文件

□置 - Doom - 灵雀云

Menu

本页概览 >

□置

目录

- □置文件
- 基础□置
- API 文档□置
- 权限说明文档□置
- 引用文档□置
  - `frontmatterMode`
- 发行说明□置
- 左导航□置
- 内部文档路由□置
- 仅包含文档路由□置
- 语言高亮插件□置
- `sites.yaml` □置
- 翻译□置
- 在代码仓库编辑文档
- 文档导出□置
- 文档检查□置
- Algolia 搜索□置
- Sitemap □置

□置文件

□置 - Doom - 灵雀云

大部分情况下，我们只需要使用静态 `yaml` □置文件即可，支持 `doom.config.yaml` 或 `doom.config.yml`，对于复杂场景，比如需要动态□置或自定义 `rspress` 插件时，可以使用 `js/ts` □置文件，支持 `.js/.ts/.mjs/.mts/.cjs/.cts` 多种文件格式。

对于 `js/ts` □置文件，我们需要导出□置，可以□合 `@alauda/doom/config` 中导出的 `defineConfig` 函数实现类型辅助：

```
import { defineConfig } from '@alauda/doom/config'

export default defineConfig({})
```

基础□置

- `lang`：默认文档语言，为方便大部分项目使用，我们默认支持中英文文档，默认语言为 `en`，如果当前文档项目不需要多语言支持，可以将此项□置为 `null` 或 `undefined`
- `title`：文档标题，会显示在浏览器标签页上
- `logo`：文档左上角 logo，支持图片链接、文件路径，绝对路径代表 `public` 目录下的文件，相对路径代表相对于当前工具目录的文件，默认使用 `doom` 包内置的 alauda logo
- `logoText`：文档标题，会显示在左上角的 logo 处
- `icon`：文档 favicon，默认同 `logo`
- `base`：文档基础路径，用于部署到非根路径，如 `product-docs`，默认为 `/`
- `outDir`：构建产物目录，默认为 `dist/{base}/{version}`，如果指定此项，则变更为 `dist/{outDir}/{version}`，其中 `version` 可选，参考多版本构建

API 文档□置

□置 - Doom - 灵雀云

```yaml
api:
  # CRD □义文件路径，相对于 doom.config.* 所在目录，支持 glob 匹配，json/yaml 文件
  crds:
    - docs/shared/crds/*.yaml
  # OpenAPI □义文件路径，相对于 doom.config.* 所在目录，支持 glob 匹配，json/yaml 文件
  openapis:
    - docs/shared/openapis/*.json
  # 渲染 openapi 相关的资源 □义时，默认会在页面内联，如果需要将相关联的资源 □义单独提取到文件中，可以配置以下选项
  # 参考 https://doom.alauda.cn/apis/references/CodeQuality.html#v1alpha1.CodeQ
  references:
    v1alpha1.CodeQualityBranch: /apis/references/CodeQualityBranch#v1alpha1.C
  # 可选，API 文档路径前缀，如果当前业务使用 gateway 等代理服务，可以配置此项
  pathPrefix: /apis
```

文档编写参考 API 文档

权限说明文档□置

```yaml
# 以下资源文件路径，相对于 doom.config.* 所在目录，支持 glob 匹配，json/yaml 文件
permission:
  functionresources:
    # `kubectl get functionresources`
    - docs/shared/functionresources/*.yaml
  roletemplates:
    # `kubectl get roletemplates -l auth.cpaas.io/roletemplate.official=true`
    - docs/shared/roletemplates/*.yaml
```

文档编写参考权限说明文档

引用文档□置

□置 - Doom - 灵雀云

```yaml
reference:
  - repo: alauda-public/product-doc-guide # 可选，引用文档仓库地址，如果不填写，则默认使用当前文档仓库地址
    branch: # [string] 可选，引用文档仓库分支
    publicBase: # [string] 可选，使用远程仓库时使用绝对路径 /images/xx.png 对应的静态资源所在目录，默认为 do
    sources:
      - name: anchor # 引用文档名称，用于在文档中引用，全局唯一
        path: docs/index.mdx#介绍 # 引用文档路径，支持锚点 □位，远程仓库相对于仓库根目录，本地相对于 doom.confi
        ignoreHeading: # [boolean] 可选，是否忽略标题，如果为 true，则不会在引用文档中显示锚点的标题
        processors: # 可选，引用文档内容处理器
          - type: ejsTemplate
            data: # ejs 模板参数，使用 `<%= data.xx %>` 访问
        frontmatterMode: merge # 可选，引用文档处理 frontmatter 模式，默认为 ignore，可选值为 ign
```

`frontmatterMode`

- `ignore`：忽略引用文档的 frontmatter，保留使用当前文档的 frontmatter
- `merge`：合并引用文档的 frontmatter，如果有相同的 key，引用文档的值会覆盖当前文档的值
- `replace`：使用引用文档的 frontmatter 替换当前文档的 frontmatter
- `remove`：移除当前文档的 frontmatter

文档编写参考引用文档

发行说明□置

```yaml
releaseNotes:
  queryTemplates:
    fixed: # 可包含 ejs 模板的 jql 语句
    unfixed:
```

release-notes.md

```
<!-- release-notes-for-bugs?template=fixed&project=DevOps -->
```

□置 - Doom - 灵雀云

release-notes.mdx

```
{/* release-notes-for-bugs?template=fixed&project=DevOps */}
```

以上述 `template=fixed&project=DevOps` 为例，`fixed` 为 `queryTemplates` 中定义的模板名称，剩余的 `query` 参数 `project=DevOps` 将作为 `ejs` ↗ 模板参数传递给 `fixed` 模板处理后作为 jira `jql` ↗ 发起 `https://jira.alauda.cn/rest/api/2/search?jql=<jql>` 请求，此 API 要求鉴权，须提供 `JIRA_USERNAME` 和 `JIRA_PASSWORD` 环境变量才能预览生效

左导航□置

```yaml
sidebar:
  collapsed: false # 可选，是否默认折叠左导航，默认折叠，文档内容不多时可以考虑设置为 false
```

内部文档路由□置

```yaml
internalRoutes: # 可选，支持 glob 匹配，相对于 docs 目录，在 cli 启用 `-i, --ignore` 选项时匹配到的路由
  - '*/internal/**'
```

仅包含文档路由□置

```yaml
onlyIncludeRoutes: # 可选，支持 glob 匹配，相对于 docs 目录，在 cli 启用 `-i, --ignore` 选项时只有此
  - '*/internal/**'
internalRoutes:
  - '*/internal/overview.mdx'
```

□置 - Doom - 灵雀云

语言高亮插件□置

```yaml
shiki:
  theme: # optional, https://shiki.style/themes
  langs: # optional, https://shiki.style/languages
  transformers: # optional, only available in js/ts config, https://shiki.sty
```

WARNING

未□置的语言将在命令行提示告警，并回退到 `plaintext` 渲染

`sites.yaml` □置

`sites.yaml` □置文件用于□置当前文档站点关联的子站点信息，引用外部站点组件和构建单版本文档时会用到此处定义的信息。

```yaml
- name: connectors # 全站唯一名称
  base: /devops-connectors # 站点访问基础路径
  version: v1.1 # 构建多版本站点时 ExternalSite/ExternalSiteLink 跳转的版本

  displayName: # 站点显示名称，如果不填写或未匹配到语言，则默认使用 name
    en: DevOps Connectors
    zh: DevOps 连接器

  # 以下属性用于构建全站点时拉取镜像，如果不填写则在最终打包完整网站时将忽略此项
  # 一般对子站点引用需要配置相关信息，对父站点引用不需要配置
  repo: https://github.com/AlaudaDevops/connectors-operator # 站点仓库地址，如果是内部 gi
  image: devops/connectors-docs # 站点构建镜像，用于构建全站点时拉取镜像
```

翻译□置

□置 - Doom - 灵雀云

```
translate:
  # 系统提示语, ejs 模板, 传入的参数有 `sourceLang`, `targetLang`, `userPrompt`, `additional
  # 其中 `sourceLang` 和 `targetLang` 是 `中文` 和 `英文` 两个字符串,
  #     `userPrompt` 为下述用户全局配置, 可能为空
  #     `additionalPrompts` 为文档 `frontmatter.i18n` 中的 `additionalPrompts` 配置,
  #     `terms` 和 `titleTranslationPrompt` 为根据文档内容包含的术语和标题动态生成的提示语, 用于让 AI 根据术
  # 默认的系统提示语如下, 可以根据实际情况进行修改
  systemPrompt: |
You are a professional technical documentation engineer, skilled in writing h

## Baseline Requirements
- Sentences should be fluent and conform to the expression habits of the <%=
- Input format is MDX; output format must also retain the original MDX format
- **CRITICAL**: Do not translate or modify ANY link content in the document.
  - URLs in markdown links: [text](URL) - keep URL exactly as is
  - Reference-style links: [text][ref] and [ref]: URL - keep both ref and URL
  - Inline URLs: https://example.com - keep completely unchanged
  - Image links:  - keep src unchanged, but alt text can be transl
  - Anchor links: [text](#anchor) - keep #anchor unchanged
  - Any href attributes in HTML tags - keep unchanged
- Do not translate professional technical terms and proper nouns, including b
- The title field and description field in frontmatter should be translated,
- Content within MDX components needs to be translated, whereas MDX component
- Do not modify or translate any placeholders in the format of __ANCHOR_N__ (
- Keep original escape characters like backslash, angle brackets, etc. unchan
- Do not add any escape characters to special characters like [], (), {}, etc
  - If source has "Architecture [Optional]", keep it as "Architecture [Option
  - If source has "Function (param)", keep it as "Function (param)" (not "Fun
  - Only add escape characters if they were present in the original text
- Preserve and do not translate the following comments, nor modify their cont
  - {/* release-notes-for-bugs */}
  - <!-- release-notes-for-bugs -->
- Remove and do not retain the following comments:
  - {/* reference-start */}
  - {/* reference-end */}
  - <!-- reference-start -->
  - <!-- reference-end -->
- Ensure the original Markdown format remains intact during translation, such
- Do not translate the content of the code block.
<% if (titleTranslationPrompt) { %>
<%- titleTranslationPrompt %>
<% } %>
<% if (terms) { %>
```

□置 - Doom - 灵雀云

```
<%- terms %>
<% } %>

<% if (userPrompt || additionalPrompts) { %>
## Additional Requirements
These are additional requirements for the translation. They should be met alo

The text for translation is provided below, within triple quotes:
"""
<% if (userPrompt) { %>
<%- userPrompt %>
<% } %>

<% if (additionalPrompts) { %>
<%- additionalPrompts %>
<% } %>
"""
<% } %>
```

在代码仓库编辑文档

```yaml
editRepoBaseUrl: alauda/doom/tree/main/docs # https://github.com/ 前缀可以省略，仅当启用
```

文档导出□置

```yaml
export:
  - name: Concepts # 可选，全局唯一 pdf 名称，默认为文档标题
    scope: '*/concepts' # 必填，字符串或数组，文档范围，支持 glob 匹配，相对于 docs 目录
```

文档检查□置

□置 - Doom - 灵雀云

```
lint:
  cspellOptions: # 可选，cspell 配置项，参考 https://github.com/streetsidesoftware/csp
```

Algolia 搜索□置

```
algolia: # 可选，Algolia 搜索配置，仅当启用 `-a, --algolia` 命令行标志符时生效
  appId: # Algolia 应用 ID
  apiKey: # Algolia API Key
  indexName: # Algolia 索引名称
```

请使用 `public/robots.txt` 进行 Algolia 爬虫验证

INFO

由于 `rspress` 当前架构限制，使用 Algolia 搜索功能需通过自定义主题 ↗ 实现，因此为统一使用相关主题功能，我们提供了 `@alauda/doom/theme` 主题入口，请添加以下主题□置文件启用：

```
export * from '@alauda/doom/theme'
```

Sitemap □置

```
siteUrl: https://docs.alauda.cn # 可选，站点 URL，用于生成 sitemap，仅当启用 `-S, --site-url
```

□定 - Doom - 灵雀云

Menu

本页概览 >

□定

目录

□录结构

元数据

排序

预览

目录结构

左侧边栏默认基于文件□录结构自动生成，一级□录中 `index` 文件即文档首页，将展示为左导航首项，子文件夹中可以使用 `index.md` 或 `index.mdx` 并定义文档一级标题来设置左侧边栏分组标题，其他子文档将自动归并到当前分组下，嵌套子文件夹也遵循相同规则。

```
├── index.md
├── start.mdx
└── usage
    ├── index.mdx
    └── convention.md
```

同时我们约定

1. `public` □录用于存放静态资源，如图片、视频等
2. `public/_remotes` 用于存放远程引用文档关联的静态资源，请勿直接依赖该□录的资源，可以将 `*/public/_remotes` 加入 `.gitignore` 避免提交到代码仓库
3. `shared` □录用于存放公共组件、可复用的文档等，不会自动生成文档数据。

□定 - Doom - 灵雀云

元数据

在文档的开头，可以通过 `frontmatter` 来定义文档的元数据，如标题、描述、作者、分类等。

```
---
title: □题
description: 描述
author: 作者
category: 分类
---
```

在文档正文中，参考 MDX 使用 `.mdx` 文件时可以使用 `frontmatter` 来访问这些元数据。

排序

除 `index.md` 或 `index.mdx` 外，其他文档将默认按照文件名排序，可以通过自定义 `frontmatter` 中的 `weight` 值来调整文档在左侧边栏中的排序（`weight` 值越小排序越靠前）。

```
---
weight: 1
---
```

WARNING

注意：□前左导航配置的变更需要重启服务才能生效，一般开发时不用过多关注。

预览

□□□□□□□□□□□□□□□□□□□□□□□□□□□ `index.mdx` □□□□□ `Overview` □□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

```
# □□

<Overview />
```

□□□□□□□□□

Markdown - Doom - □雀云

Menu

本页概览 ›

Markdown

□了标准的 gfm↗ 语法外，Doom 内置了一些额外的 Markdown 扩展功能。

□录

Callouts

Mermaid

属性扩展

Callouts

源码标注组件

NOTE

1. 请根据实际语言使用行内代码注释，如 `;`, `%`, `#`, `//`, `/** */`, `--` 和 `<!-- -->` 等
2. 如果需要将其视为代码注释，请使用 `[\!code callout]` 进行转义
3. 有时，`:::callouts` 由于嵌套缩进导致解析显示异常，可以使用 `<div class="doom-callouts">` 或 `<Callouts>` 组件代替

```sh
Memory overhead per virtual machine ≈ (1.002 × requested memory) \
              + 218 MiB \  # [\!code callout]
              + 8 MiB × (number of vCPUs) \  # [\!code callout]
              + 16 MiB × (number of graphics devices) \  # [\!code callout]
              + (additional memory overhead) # [\!code callout]
```

```
:::callouts

1. Required for the processes that run in the `virt-launcher` pod.
2. Number of virtual CPUs requested by the virtual machine.
3. Number of virtual graphics cards requested by the virtual machine.
4. Additional memory overhead:
   - If your environment includes a Single Root I/O Virtualization (SR-IOV) n
   - If Secure Encrypted Virtualization (SEV) is enabled, add 256 MiB.
   - If Trusted Platform Module (TPM) is enabled, add 53 MiB.

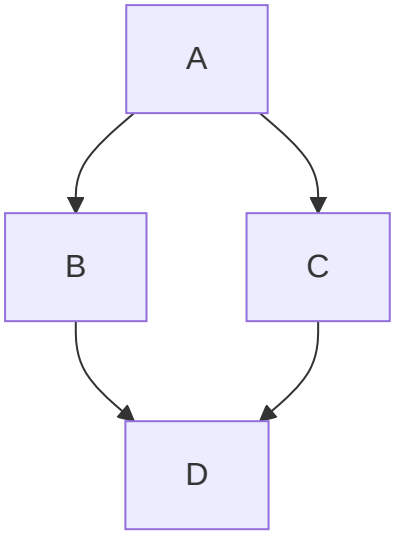
:::
```

```
Memory overhead per virtual machine ≈ (1.002 × requested memory) \
              + 218 MiB \   1
              + 8 MiB × (number of vCPUs) \   2
              + 16 MiB × (number of graphics devices) \   3
              + (additional memory overhead)   4
```

1. Required for the processes that run in the `virt-launcher` pod.
1. Number of virtual CPUs requested by the virtual machine.
1. Number of virtual graphics cards requested by the virtual machine.
1. Additional memory overhead:

   - If your environment includes a Single Root I/O Virtualization (SR-IOV) network device or a Graphics Processing Unit (GPU), allocate 1 GiB additional memory overhead for each device.
   - If Secure Encrypted Virtualization (SEV) is enabled, add 256 MiB.
   - If Trusted Platform Module (TPM) is enabled, add 53 MiB.

□□□□□□□□□□□□ Shiki Transformers ↗□



Markdown - Doom - □雀云

Mermaid ↗

图表绘制工具

```
```mermaid
graph TD;
    A-->B;
    A-->C;
    B-->D;
    C-->D;
```
```

A
B
C
D

配合 Markdown Preview Mermaid ↗ 可以在 VSCode 中实时预览。

属性扩展

WARNING

当前仅支持在 `.md` 文件中使用，参考相关 issue ↗

```
{width="100" height="100"}
```

MDX - Doom - □雀云

Menu 本页概览 >

MDX

MDX ↗ □一种 Markdown 的扩展语法，允许在 Markdown 中使用 JSX 语法，使用方式可以参考 rspress MDX ↗ 。

□录

- rspress 组件
- doom 组件
  - `Overview`
  - `Directive`
  - `ExternalSite`
  - `ExternalSiteLink`
  - `AcpApisOverview` 与 `ExternalApisOverview`
  - Term
    - `props`
  - `TermsTable`
    - `props`
  - `JsonViewer`
- 自定义组件复用

---

rspress 组件

`rspress` 主题提供的内置组件↗大部分已调整为全局组件，可以在 `.mdx` 文件中无需导入直接使用，包括：

- `Badge`

MDX - Doom - □雀云

- `Card`
- `LinkCard`
- `PackageManagerTabs`
- `Steps`
- `Tab/Tabs`
- `Toc`

其他不常用的组件可以通过 `@rspress/core/theme` 导入使用，例如：

preview.mdx

```
import { SourceCode } from '@rspress/core/theme'

<SourceCode href="/" />
```

---

doom 组件

`doom` 提供了一些全局组件来辅助文档编写，不需要导入即可直接使用，目前包括：

`Overview`

文档概览组件，用于展示文档目录

`Directive`

有时，由于嵌套缩进，自定义容器↗语法可能失效，可以使用 `Directive` 组件代替

```
- □语言文档 (`doc/en`)的目录结构需要与 `doc/zh` 目录下的文档完全一致，保证 □语言文档的链接除了语言标识外完全相同。

  <Directive type="danger" title="注意">
    如果是使用自动化翻译工具进行翻译，则无需关心该问题，自动化翻译工具会自动根据
    `doc/zh` 生成目标语言文档的目录结构。
  </Directive>
```

MDX - Doom - □雀云

- 多语言文档(`doc/en`)的目录结构需要与 `doc/zh` 目录下的文档完全一致，保证多语言文档的链接除了语言标识外完全相同。

> **注意**
>
> 如果□使用自动化翻译工具进行翻译，则无需关心该问题，自动化翻译工具会自动根据 `doc/zh` 生成目标语言文档的目录结构。

`ExternalSite`

引用外部站点组件

```
<ExternalSite name="connectors" />
```

> **Note**
>
> 因为 DevOps 连接器的发版周期与灵雀云容器平台不同，所以 DevOps 连接器的文档现在作为独立的文档站点托管在 DevOps 连接器↗。

`ExternalSiteLink`

引用外部站点链接组件

```
<ExternalSiteLink name="connectors" href="link.mdx#hash" children="Content" /
```

Content ↗

> **TIP**
>
> 在 mdx 中 `<ExternalSiteLink name="connectors" href="link" children="Content" />` 与下面的内容含义不同

```
<ExternalSiteLink name="connectors" href="link">
  Content {/* □□□□ `p` □□□ */}
</ExternalSiteLink>
```

□□□□□□□□□□□ `p` □□□□□□□□□□□□□□□□□□ `children` □□□□

`AcpApisOverview` □ `ExternalApisOverview`

□□□□□□ API □□□□

```
<AcpApisOverview />
{/* same as following */}
<ExternalApisOverview name="acp" />

<ExternalApisOverview name="connectors" />
```

Note

□□ ACP APIs □□□□□□□□□□□ ACP APIs □□↗□

Note

□□ DevOps □□□ APIs □□□□□□□□□□□ DevOps □□□ APIs □□↗□

Term

□□□□□□□□□□□□□□□□

```
<Term name="company" textCase="capitalize" />
<Term name="product" textCase="lower" />
<Term name="productShort" textCase="upper" />
```

□□□ □□□□□□□□ ACP

`props`

- `name`：□□□□□□□□□□□□□□□□□□
- `textCase`：□□□□□□□□□□□□□ `lower`，`upper`，`capitalize`

`TermsTable`

□□□□□□□□□□□

```
<TermsTable />
```

□□	□□	□□□□	□□	□□□□	□□
company	□□□	-	Alauda	-	□□□□
product	□□□□□□□	-	Alauda Container Platform	-	□□□□
productShort	ACP	-	ACP	-	□□□□□□

`props`

- `terms`：`NormalizedTermItem[]` □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

`JsonViewer`

```
<JsonViewer value={{ key: 'value' }} />
```

yaml　　json

```yaml
key: value
```

□□□□□□□□

□□□□□□□□□□□□□□□□□□□□□□ `shared` □□□□□□□□□□□□□□□□□□□□□□□□

```
import CommonContent from './shared/CommonContent.mdx'

<CommonContent />
```

□□□□□□□□□ runtime↗ □□□ API□□□□□□ `.jsx/.tsx` □□□□□□□□□ `.mdx` □□□□□□□□□

```
// shared/CommonContent.tsx
export const CommonContent = () => {
  const { page } = usePageData()
  return <div>{page.title}</div>
}

// showcase/content.mdx
import { CommonContent } from './shared/CommonContent'
;<CommonContent />
```

WARNING

□□□□□ `.mdx` □□□□□□□□□ `props` □□□□□□ issue↗□□□□□□□□ `props` □□□□□□ `.jsx/.tsx` □□□□□□

国际化 - Doom - 灵雀云

Menu 本页概览 >

国际化

`alauda` 内部大部分文档都是中英文双语，因此我们默认支持使用 `en` / `zh` 两个子文件夹来存放不同语言的文档，推荐在 `public` 目录下也按 `en` / `zh` 子文件夹存放静态资源，这样可以方便文档内容和静态资源的管理。

目录

- `i18n.json`
- `.ts/.tsx`
- `.mdx`

`i18n.json`

对于可复用组件，如果需要在同一个组件中同时支持中英文，那么就需要先在 `docs` 目录下创建 `i18n.json` 文件，然后在组件中通过 `useI18n` 来获取当前语言的文本，比如：

docs/i18n.json

```json
{
  "title": {
    "zh": "标题",
    "en": "Title"
  },
  "description": {
    "zh": "描述",
    "en": "description"
  }
}
```

.ts/.tsx

```
import { useI18n } from '@rspress/runtime'

export const CommonContent = () => {
  const t = useI18n()
  return <h1>{t('title')}</h1>
}
```

.mdx

```
import { useI18n } from '@rspress/runtime'

# {useI18n()('title')}

{useI18n()('description')}
```

API □档 - Doom - 灵雀云

Menu 本页概览 >

API □档

□据实际业务，我们一般会将 API 分为高级 API 和 CRD (Custom Resource Definition) 两种，因此在目录结构上一般分为：

```
├── apis
│   ├── advanced-apis # □级 API
│   ├── crds # CRDs
│   └── references # 公共引用
```

目录

- 高级 API
  - `props`
- CRD
  - `props`
- 公共引用
  - `props`
- 指定 openapi 路径

高级 API

advanced-apis/codeQualityTaskSummary.mdx

API 文档 - Doom - 灵雀云

```
# CodeQualityTaskSummary

<OpenAPIPath path="/plugins/v1alpha1/template/codeQuality/task/{task-id}/summ
```

参考 CodeQualityTaskSummary。

`props`

- `path`: OpenAPI schema `paths` 下的路径
- `pathPrefix`: 可以用于覆盖全局配置中的 `api.pathPrefix`
- `openapiPath`: 参考指定 openapi 路径

CRD

crds/ArtifactCleanupRun.mdx

```
# ArtifactCleanupRun

<K8sCrd name="artifactcleanupruns.artifacts.katanomi.dev" />
```

参考 ArtifactCleanupRun。

`props`

- `name`: CRD `metadata.name`
- `crdPath`: 类似指定 openapi 路径，用于指定特定的 CRD 文件

公共引用

references/CodeQuality.mdx

```
# CodeQuality

<OpenAPIRef schema="v1alpha1.CodeQuality" />
```

□□ CodeQuality□

`props`

- `schema`: OpenAPI schema `definitions` (v2) or `component/schemas` (v3) □□□□
- `openapiPath`: □□□□ openapi □□

□□ openapi □□

□□ `OpenAPIPath` □ `OpenAPIRef` □□□□□□□□□ openapi □□□□□□□□□□□□□□□□□□□□□□□ openapi □□□□□□□ `openapiPath` □□□□□

```
<OpenAPIPath
  path="/plugins/v1alpha1/template/codeQuality/task/{task-id}/summary"
  openapiPath="shared/openapis/katanomi.json"
/>
```

□限说明文档 - Doom - 灵雀云

Menu

本页概览 >

权限说明文档

```
<K8sPermissionTable functions={['devops-testplans', 'devops-testmodules']} />
```

目录

`props`

□例

`props`

- `functions`: `string[]` - 必填，需要展□的 `FunctionResource` 资源名称数组

示例

□能	操作	平台管理员	平台审计人员	项目管理员	命名空间管理员	开发人员	集群管理员
测试计划 `devops-testplans`	查看	✓	✓	✓	✓	✓	✕
	创建	✓	✕	✓	✓	✓	✕
	更新	✓	✕	✓	✓	✓	✕
	删除	✓	✕	✓	✓	✓	✕

□限说明文档 - Doom - 灵雀云

□能	操作	平台管理员	平台审计人员	项目管理员	命名空间管理员	开发人员	集群管理员
测试模块 `devops-testmodules`	查看	✓	✓	✓	✓	✓	✕
	创建	✓	✕	✓	✓	✓	✕
	更新	✓	✕	✓	✓	✓	✕
	删除	✓	✕	✓	✓	✓	✕

□用文档 - Doom - 灵雀云

Menu

本页概览 >

□用文档

□ Markdown 文件中:

```
<!-- reference-start#name -->

<!-- reference-end -->
```

□ MDX 文件中:

```
{/* reference-start#name */}

{/* reference-end */}
```

上述 `name` 为引用文档的名称，参考引用文档配置，如果引用的文档内容使用了远程仓库图片静态资源，相关静态资源将自动存储□本地 `<root>/public/_remotes/<name>` 目录下。

以下为使用 `<!-- reference-start#ref -->` 的实例:

目录

引用文档配置

`frontmatterMode`

□用文档配置

□用文档 - Doom - 灵雀云

```yaml
reference:
  - repo: alauda-public/product-doc-guide # □选，引用文档仓库地址，如果不填写，则默认使用当前文档仓库地址
    branch: # [string] □选，引用文档仓库分支
    publicBase: # [string] □选，使用远程仓库时使用绝对路径 /images/xx.png 对应的静态资源所在目录，默认为 do
    sources:
      - name: anchor # 引用文档名称，用于在文档中引用，全局唯一
        path: docs/index.mdx#介绍 # 引用文档路径，支持锚点定位，远程仓库相对于仓库根目录，本地相对于 doom.confi
        ignoreHeading: # [boolean] □选，是否忽略标题，如果为 true，则不会在引用文档中显示锚点的标题
        processors: # □选，引用文档内容处理器
          - type: ejsTemplate
            data: # ejs 模板参数，使用 `<%= data.xx %>` 访问
        frontmatterMode: merge # □选，引用文档处理 frontmatter 模式，默认为 ignore，□选值为 ign
```

`frontmatterMode`

- `ignore`：忽略引用文档的 frontmatter，保留使用当前文档的 frontmatter
- `merge`：合并引用文档的 frontmatter，如果有相同的 key，引用文档的值会覆盖当前文档的值
- `replace`：使用引用文档的 frontmatter 替换当前文档的 frontmatter
- `remove`：移除当前文档的 frontmatter

文档编写参考引用文档

□署 - Doom - 灵雀云

Menu

本页概览 >

□署

目录

□建与预览

多版本□建

合并目录结□

动态挂载配置文件

构建与预览

在部署之前，我们需要先对项目进行生产环境的□建，并在本地进行预览，以确保项目能够正常运行：

```
doom build # □建静态产物
doom serve # 以生产模式预览 □建产物
```

多版本构建

默认情况下，`doom build` 会将□建产物输出到 `dist` 目录，如果需要□建多个版本的文档，可以通过 `-v` 参数指定版本号，例如：

□署 - Doom - 灵雀云

```
# 一般由分支名确定，如 release-4.0 对应 4.0 版本
doom build -v 4.0 # □建 4.0 版本，产物输出到 dist/4.0，文档访问路径为 {base}/4.0
doom build -v master # □建 master 版本，产物输出到 dist/master，文档访问路径为 {base}/master
doom build -v {other} # □建其他版本，产物输出到 dist/{other}，文档访问路径为 {base}/{other}

# unversioned 和 unversioned-x.y 为特殊版本号，用于 □建无版本号前缀的文档
doom build -v unversioned # □建无版本号前缀的文档，产物输出到 dist/unversioned，文档访问路径为 {base}
doom build -v unversioned-4.0 # □建无版本号前缀但导航栏展示版本号 4.0 的文档，产物输出到 dist/unversio
```

合并目录结构

```
|── console-platform
|   ├── 4.0
|   ├── 4.1
|   ├── index.html
|   ├── overrides.yaml
|   └── versions.yaml
|── console-devops-docs
|   ├── 4.0
|   ├── 4.1
|   ├── index.html
|   ├── overrides.yaml
|   └── versions.yaml
|── console-tekton-docs
|   ├── 1.0
|   ├── 1.1
|   ├── index.html
|   ├── overrides.yaml
|   └── versions.yaml
```

index.html

□署 - Doom - 灵雀云

```html
<!DOCTYPE html>
<html>
  <head>
    <title>Redirecting...</title>
    <meta http-equiv="refresh" content="0; url=/console-docs/4.1" />
  </head>
  <body>
    <p>Redirecting to <a href="/console-docs/4.1">/console-docs/4.1</a></p>
  </body>
</html>
```

动态挂载配置文件

overrides.yaml

```yaml
# 文档信息，每个文档都可以挂载覆盖默认配置
title:
  en: Doom - Alauda
  zh: Doom - 灵雀云
logoText:
  en: Doom - Alauda
  zh: Doom - 灵雀云
```

versions.yaml

```yaml
- '4.1'
- '4.0'
```